



Práctica 3: TAD AB y TAD ABEnteros

OBJETIVOS

- Diseñar, implementar y usar los TAD AB y ABEnteros.
- Comprender cómo se trabaja con árboles binarios (AB) y con árboles binarios de búsqueda (ABB).

(Lee bien el enunciado completo de la práctica ANTES de llevar a cabo la resolución de la misma)

1) TAD AB:

- Realiza la **especificación lógica** del **TAD Árbol Binario (AB)** siguiendo las distintas etapas de la “técnica de abstracción de datos” que hemos estudiado. Debes hacer un estudio previo para determinar qué operaciones son básicas en un AB (*revisa los apuntes de clase*).
- Implementa el TAD AB :
 - Paquete modelos: Interfaz I_AB
 - Paquete jerárquicos: Clase NodoAB (nodo), Clase AB (árbol binario)
- Enriquece el TAD AB** considerando la operación **EliminarNodosInferiores**, que elimine los nodos que se encuentran en niveles inferiores a un nivel dado (n).

Para ello:

- Actualiza la especificación lógica del TAD AB e incluye la nueva operación **EliminarNodosInferiores()** en la interfaz del AB. Por último, implementa este método recursivo en la clase AB.
- El método lanzadera debe tener como único parámetro un número entero (el nivel n que vamos a considerar).

2) TAD ABEnteros:

- Realiza la **especificación lógica** del **TAD Árbol Binario de claves enteras (ABEnteros)** siguiendo las distintas etapas de la “técnica de abstracción de datos” que hemos estudiado. Debes hacer un estudio previo para determinar qué operaciones son básicas en un ABEnteros.
- Implementa el TAD ABEnteros haciendo uso del TAD AB anterior (será un AB pero considerando solo claves enteras (Integer))
 - Paquete modelos: Interfaz I_ABEnteros
 - Paquete jerárquicos: Clase ABEnteros



c) **Enriquece el TAD ABEnteros** considerando las siguientes operaciones:

c.1) Método recursivo booleano **EsABB()** que devuelva true si el AB de claves enteras considerado es ABB y false en caso contrario.

c.2) Método recursivo booleano **RaizIgualNodosInternos()** que trabaje sobre un AB de claves enteras y devuelva true si el valor de la clave contenida en el nodo raíz del AB es igual al número de nodos internos del árbol, en caso contrario, devolverá false.

c.3) Método recursivo **MinimoValorNivel()** que calcule el valor mínimo que existe en un nivel dado (n) en un AB de claves enteras.

- Antes de invocar al método debes comprobar que existen al menos n niveles en el árbol.
- Para ello, implementa un nuevo método que calcule el número de niveles que tiene el árbol (método **NivelesAB()**). El algoritmo debe contemplar la posibilidad de que no existan n niveles en el árbol en cuyo caso se lanzará un mensaje del tipo “ERROR: no existen 4 niveles en el árbol. El árbol tiene 3 niveles”. (Ten en cuenta que consideramos que el nivel del nodo raíz es el nivel 0)

Cuestión 1

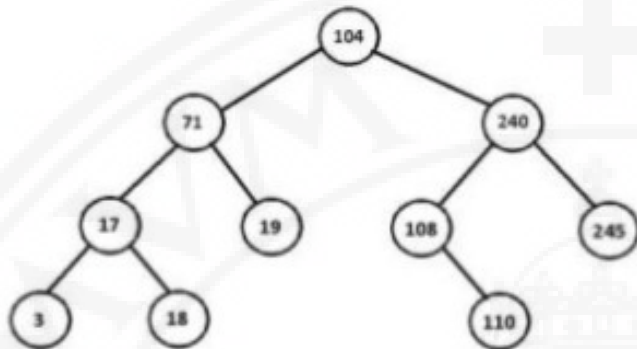
- ¿En cuál de los TADs diseñados (TAD AB, TAD ABEnteros) debe ir el método NivelesAB() si consideramos que se trata de una operación básica? ¿Por qué?

A tener en cuenta en la implementación de los TADs:

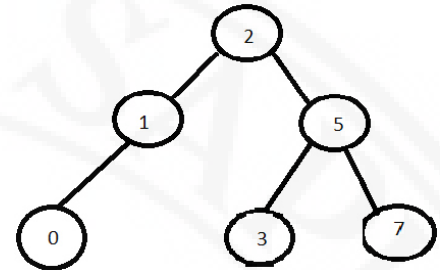
- Considera la posibilidad de aplicar herencia entre interfaces y herencia entre clases y justifica en la memoria de la práctica la decisión que has tomado al respecto al realizar el diseño de las interfaces y clases consideradas.
- Recuerda que en la memoria de la práctica debes incluir una breve descripción del diseño que has realizado de cada uno de los métodos marcados en rojo (idea que has desarrollado).



3) Uso de los TADs AB y ABEnteros:



Árbol AB1



Árbol AB2

➤ Implementa la **clase PruebaABEnteros** donde llevarás a cabo las siguientes tareas:

- Crear los árboles AB1 y AB2 anteriores (métodos estáticos de esta clase **CrearAB1()**, **CrearAB2()**)
- A continuación, de forma repetitiva, mostrar en pantalla el siguiente menú de opciones hasta que el usuario elija la opción 0, momento en el cuál, se mostrará un mensaje de despedida ("Gracias por utilizar nuestra aplicación...") y finalizará la ejecución del programa.

MENÚ PRINCIPAL

1. Listado de las claves del AB1 en PreOrden
2. Listado de las claves del AB1 en InOrden
3. Listado de las claves del AB1 en InOrden Converso
4. Listado de las claves del AB1 en PostOrden
5. Listado de las claves del AB2 en InOrden
6. Comprobar si los árboles AB1 y AB2 son ABB
7. Comprobar Raiz Igual a Nodos Internos
8. Eliminar en AB1 nodos por debajo del nivel 2
9. Eliminar en AB2 nodos por debajo del nivel 3
10. Comprobar mínimo valor de un nivel
0. Salir



La tarea a realizar en cada una de las opciones anteriores se describe a continuación:

1. Mostrar un listado en pantalla de las claves del AB1 en PreOrden
2. Mostrar un listado en pantalla de las claves del AB1 en InOrden
3. Mostrar un listado en pantalla de las claves del AB1 en InOrden Converso
4. Mostrar un listado en pantalla de las claves del AB1 en PostOrden
5. Mostrar un listado en pantalla de las claves del AB2 en InOrden
6. Comprobar si los árboles AB1 y AB2 son ABB y mostrar en pantalla el resultado obtenido en cada caso.
7. Ejecutar el método RaizIgualNodosInternos() sobre el árbol AB1 y sobre el árbol AB2 y mostrar en pantalla el resultado obtenido en cada caso.
8. Eliminar en el árbol AB1 los nodos que se encuentran por debajo del nivel 2. Después, realizar un listado de claves en InOrden de AB1.
9. Eliminar en el árbol AB2 los nodos que se encuentran por debajo del nivel 3. Después, realizar un listado de claves en InOrden de AB2.
10. Pedir al usuario un número entero (nivel n) e invocar al método MinimoValorNivel() mostrando finalmente el resultado obtenido. Los mensajes que se mostrarán por pantalla tendrán un formato similar a los que aparecen en los siguientes ejemplos:
 - i. *“Árbol AB1: El valor mínimo obtenido en el nivel N es XX”*
 - ii. *“Árbol AB2: El valor mínimo obtenido en el nivel M es YY”*
 - iii. *“ERROR: no existen N niveles en el árbol. El árbol tiene Z niveles”*.

MUY IMPORTANTE: Cuida especialmente la interfaz con el usuario. Haz una pausa en ejecución cada vez que sea necesario ("Pulse una tecla para continuar...")

Y no olvides mostrar una cabecera al ejecutar cada una de las opciones del menú, por ejemplo:

LISTADO DE CLAVES DEL AB1 EN PREORDEN

=====

** Para facilitar las pruebas, se proporcionan a continuación la secuencia de claves obtenidas al realizar los diferentes recorridos en profundidad sobre los árboles AB1 y AB2 antes de ser modificados:

Árbol AB1

Recorrido en PreOrden : 104,71,17,3,18,19,240,108,110,245

Recorrido en InOrden : 3,17,18,71,19,104,108,110,240,245



Recorrido en InOrden Converso: 245,240,110,108,104,19,71,18,17,3

Recorrido en PostOrden : 3,18,17,19,71,110,108,245,240,104

Árbol AB2

Recorrido en InOrden : 0,1,2,3,5,7

Cuestión 2

Si hubiésemos implementado los métodos de esta práctica en un ABB .. ¿En qué medida cambiaría el código fuente que has desarrollado?.. ¿Por qué?.. Justifica de forma adecuada tu respuesta (*No es preciso que realices la implementación del TAD ABB*).

CONTENIDO DE LA MEMORIA A ENTREGAR CON LA PRÁCTICA:

- **Portada que incluya:**
 - * Título de la práctica
 - * N° de Grupo de prácticas
 - * Fecha de entrega
 - * Alumnos
 - * Turno y Profesor de Laboratorio (Nuria/ Jaime)
- **Especificación lógica completa** de los TAD diseñados.
- **Breve descripción de los procesos** llevados a cabo en los distintos métodos que implementan cada uno de los procesos indicados con anterioridad.
- Listado de todo el **código fuente** desarrollado.
- **Pruebas de ejecución.**
- **Respuesta razonada a las cuestiones 1 y 2** planteadas

Importante:

- La memoria de la práctica y el código fuente de la misma deberán ser entregados a través del campus virtual de acuerdo a las instrucciones que allí se detallan.
- No se admitirán entregas por otra vía distinta (correo electrónico, drive,... etc.).
- No se admitirá entregas fuera del plazo establecido.
- Se recuerda que el profesor de laboratorio podrá solicitar que los alumnos lleven a cabo una defensa presencial de la práctica si lo considera necesario.

FECHAS A CONSIDERAR:

Publicación de la práctica: Jueves, 24 de Abril de 2026

Entrega de la práctica : **Domingo, 24 de Mayo de 2026 a las 22 h.** (entrega en el campus virtual)